

Практическая работа «Технологии распознавания двумерных штрих-кодов»

Цель: изучить основные принципы работы технологий распознавания двумерных штрих-кодов, а также приобрести практические навыки применения двумерных штрих-кодов для хранения и передачи информации.

Теоретическая часть

На настоящий момент существует большое количество двумерных штриховых кодов (бар-кодов), наиболее популярными среди которых являются Aztec Code, DataMatrix, PDF417 и QR Code. Примеры кодов, содержащих одну и ту же фразу «Test code» представлены на рисунке 1.1.

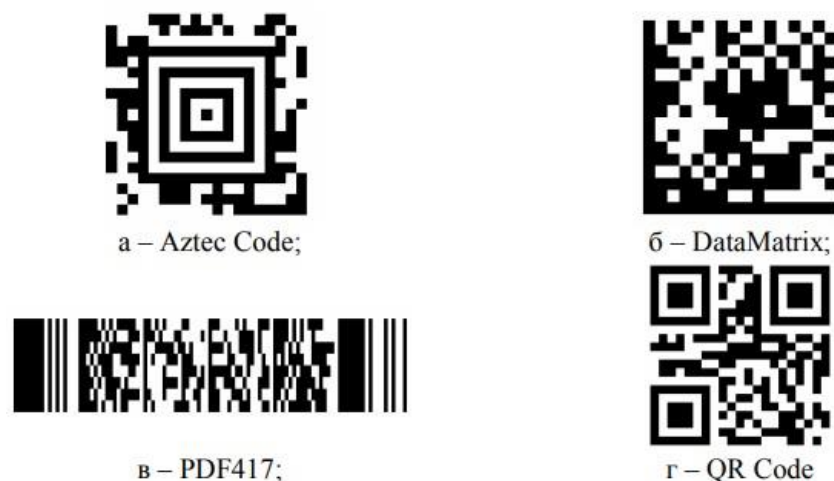


Рисунок 1.1 – Примеры кодирования фразы «Test code»

У каждого из этих кодов есть свои достоинства и недостатки, что позволяет использовать их в различных условиях. Центральная мишень Aztec Code позволяет распознавать его в условиях пересечения других изображений с краями кода. DataMatrix – хорошо зарекомендовавший себя код, но требующий свободное место по краям. PDF417 сделан по подобию одномерных штрих кодов для облегчения сканирования, но может кодировать меньше информации по сравнению с другими двумерными бар-кодами. Популярный QR Code является одним из наиболее оптимальных кодов, хотя при малых размерах кода квадраты-мишени отнимают много места. Некоторые из кодов защищены стандартами и патентами, а некоторые переданы в общественное достояние.

В отличие от распознавания одномерных кодов, где необходимо прочесть и декодировать штриховую линию, для двумерных кодов необходимо чётко определить не только границы, но и некоторые синхронизирующие элементы. Т.к. в большинстве случаев двумерные бар-коды представляют собой квадраты или прямоугольники, то необходимо выделить четыре угловые точки. Большинство алгоритмов включают в себя следующие этапы:

- 1- предварительная обработка изображения;
- 2- детектирование углов кода;
- 3- обратное перспективное преобразование;
- 4- получение матрицы кода и декодирование.

На рисунке 1.2 слева показано правильное расположение двумерного бар-кода при кодировании, а справа – пример того, как может находиться код на входном изображении. Обратное перспективное преобразование – это перевод изображения к тому, что было раньше.

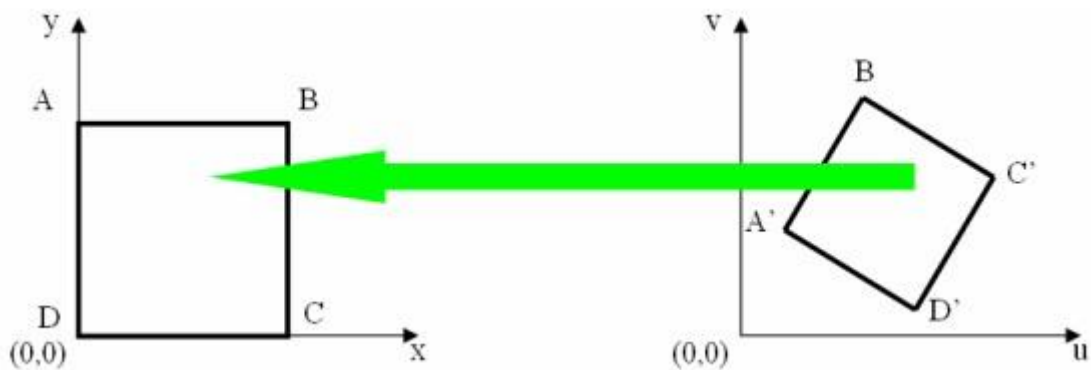


Рисунок 1.2 – Схема обратного перспективного преобразования

После того, как точки получены, можно приступить к наложению матрицы распознавания. Так при восстановлении матрицы кода «DataMatrix» нужно построить на полученных линиях сетку, и определить значения ячеек. Однако, поскольку размеры кода могут быть большими, то из-за наклона камеры возможно изменение размера ячеек. Поэтому по размерам ячеек граничных пунктирных линий определяется зависимость их изменений с использованием метода наименьших квадратов. После этого строится сетка с различными значениями размеров ячеек, представленных на рисунке 1.3.

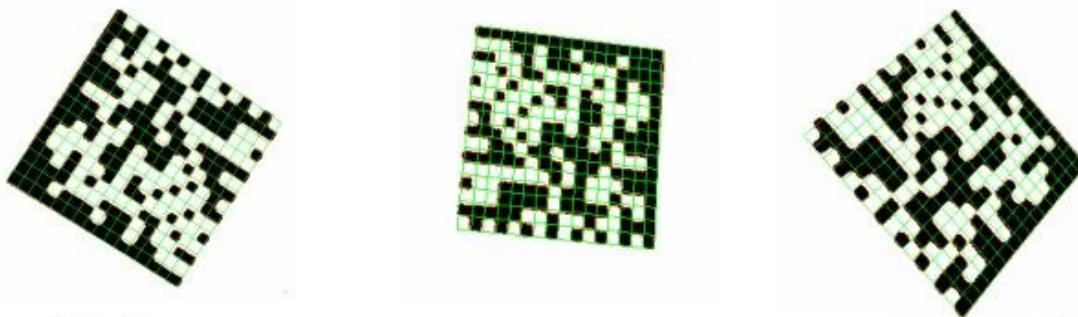


Рисунок 1.3 – Полученная сетка матрицы для различных положений кода DataMatrix

Полученная матрица кодируется определенным образом, причем в нее добавляются корректировочные коды Рида-Соломона. Т.е. часть информации несет в себе смысл, а другая часть – позволяет восстановить эту информацию при повреждении матрицы. Для восстановления данных необходимо знать правила кодирования конкретного кода. Ниже приведена информация кодирования кода «Aztec». Важнейшим элементом при детектировании кода является центральная мишень с битами ориентации, а также синхронизирующие пунктирные линии, представленные на рисунке 1.4.

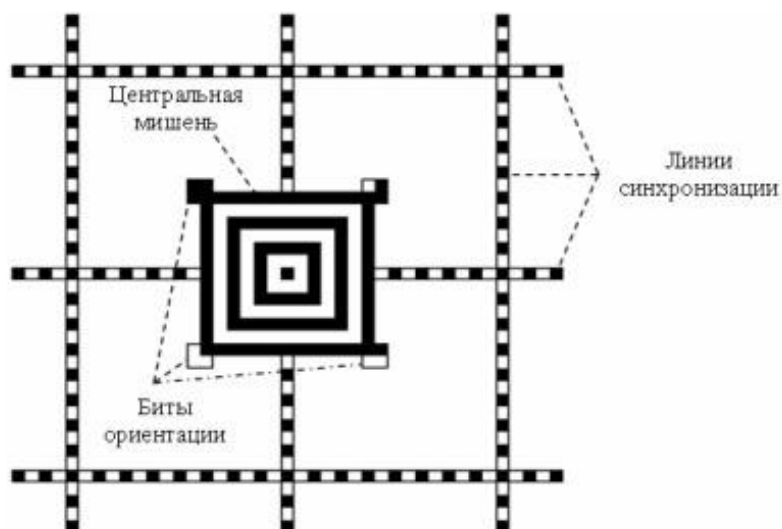


Рисунок 1.4 – Элементы для нахождения, ориентации и синхронизации кода Aztec

Наличие центральной мишени позволяет легко детектировать код, выяснить ориентацию с помощью элементов по краям и синхронизировать распознавание кода по соответствующим линиям.

Кодируемые данные располагаются по слоям (уровням), которые обвиваются вокруг центральной мишени по так называемому принципу «домино» по часовой стрелке (см. рисунок 1.5).

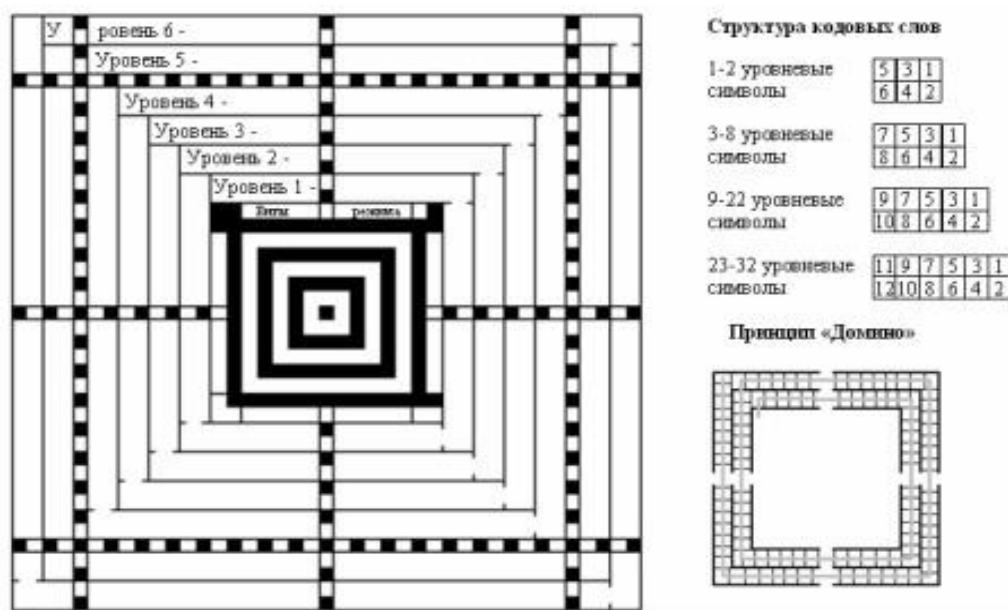


Рисунок 1.5 – Расположение данных, структура кодовых слов и принцип «домино»

Информация о режиме кодирования находится между ориентировочными элементами мишени по принципу «домино» и содержит в себе 40 бит. Уровни шириной в два бита и с различной длиной кодовых слов располагаются по принципу «домино», исключая синхронизирующие элементы – пунктирные линии сетки, которых при увеличении количества уровней становится больше. Центральные пунктирные линии присутствуют в любом стандартном коде Aztec. Остальные линии добавляются между 4 и 5 слоями, в слоях 12 и 27, разделяя кодовые слова.

В начальном уровне #1 содержится 128 бит информации, а в последующих добавляется по 32 бита, поэтому общее количество бит информации в символе длины L может быть рассчитано по формуле:

$$Cb = (112 + 16 * L) * L \quad (1.1)$$

Коррекция Рида-Соломона осуществляется для кодовых слов различной длины:

- в 1-2 уровнях символах используются 6-битные кодовые слова;
- в 3-8 уровнях символах используются 8-битные кодовые слова;
- в 9-22 уровнях символах используются 10-битные кодовые слова;
- в 23-32 уровнях символах используются 12-битные кодовые слова.

Поэтому количество кодовых слов, о которых говорится в блоке данных, рассчитывается следующим образом:

$$Cw = Cb \div K \quad (1.2)$$

где K – количество бит в кодовом слове;

div – функция целого деления.

Возможные размеры символов Aztec Code представлены в таблице 1.1.

Таблица 1.1 – Размеры и вместимость символов «Aztec Code»

Уровни	Размер	Вместимость	Уровни	Размер	Вместимость	Уровни	Размер	Вместимость
1	19 × 19	21 × 6	12	67 × 67	364 × 10	23	113 × 113	920 × 12
2	23 × 23	48 × 6	13	71 × 71	416 × 10	24	117 × 117	992 × 12
3	27 × 27	60 × 8	14	75 × 75	470 × 10	25	121 × 121	1066 × 12
4	31 × 31	88 × 8	15	79 × 79	528 × 10	26	125 × 125	1144 × 12
5	37 × 37	120 × 8	16	83 × 83	588 × 10	27	131 × 131	1224 × 12
6	41 × 41	156 × 8	17	87 × 87	652 × 10	28	135 × 135	1306 × 12
7	45 × 45	196 × 8	18	91 × 91	720 × 10	29	139 × 139	1392 × 12
8	49 × 49	240 × 8	19	95 × 95	790 × 10	30	143 × 143	1480 × 12
9	53 × 53	230 × 10	20	101 × 101	894 × 10	31	147 × 147	1570 × 12
10	57 × 57	272 × 10	21	105 × 105	940 × 10	32	151 × 151	1664 × 12
11	61 × 61	316 × 10	22	109 × 109	1020 × 10			

В таблице показаны физические размеры символов и их вместимость (количество кодовых слов умноженных на количество бит). Информация о режиме кодирования, как отмечалось выше, содержит 40 бит, из которых только 16 бит содержат полезную информацию, остальные 24 бита – выполняют функцию коррекции ошибок Рида-Соломона. Первые 5 бит содержат размер символа, который определяется через количество слоёв минус 1. Т.е. если первые 5 бит – нулевые, то код содержит только 1 уровень. Следующие 11 бит кодируют длину сообщения, которая определяется как количество кодовых слов минус 1, включая символы следующие за кодовыми словами для контроля ошибок.

Эти 16 служебных бит расположены в 4-битных словах, а остальные 6 проверочных слов добавляются кодированием Рида-Соломона с полем Галуа GF(16), основанным на главном

полиноме модуля x^4+x+1 (коэффициент генерации полинома= 19).

Порождающий полином $(x-21) \cdot (x-26)$ следующий:

$$x^6 + 7x^5 + 9x^4 + 3x^3 + 12x^2 + 10x + 12. \quad (1.3)$$

Все десять слов следуют друг за другом, начиная с верхнего левого угла. Информационное сообщение (закодированное в уровнях символа) кодируется по-разному в зависимости от размеров. В таблице 1.2 показаны характеристики распределения Рида-Соломона, необходимые для кодирования-декодирования информации.

Таблица 1.2 – Размеры кодовых слов и главные полиномы модуля

Уровни	Размер кодового слова	Поле Галуа	Полином	Коэффициент генерации
1 – 2	6 бит	GF(64)	$x^6 + x + 1$	67
3 – 8	8 бит	GF(256)	$x^6 + x^5 + x^3 + x^2 + 1$	301
9 – 22	10 бит	GF(1024)	$x^{10} + x^3 + 1$	1033
23 – 32	12 бит	GF(4096)	$x^{12} + x^6 + x^5 + x^3 + 1$	4201

Высокоуровневое кодирование состоит из двух этапов. На первом этапе, строка символов преобразуются в поток бит. На втором этапе поток бит подвергается преобразованию, которое предназначено для того, чтобы исключить длительного повторения одинаковых бит.

На этапе 1 берутся символы сообщения, которые кодируются с использованием таблицы 1.3.

Размеры символов могут быть 4, 5 и 8 бит – соответственно для цифр (Digit), остальных режимов таблицы 1.3 и побайтного кодирования.

Сообщение начинает кодироваться в режиме Upper и может быть переключено в другой режим с помощью UL (Upper), LL (Lower), ML (Mixed), PL (Punct), DL (Digit) или переведено в режим только для ввода единственного символа с помощью US (Upper) и PS (Punct). При переключении на режим Digit происходит смена количества бит на символ с 5 на 4.

Таблица 1.3 – Высокоуровневое кодирование «Aztec Code»

Value	Upper		Lower		Mixed		Punct		Digit	
	Char	ASCII	Char	ASCII	Char	ASCII	Char	ASCII	Char	ASCII
0	PS		PS		PS		FLG(n)	***	PS	
1	SP	32	SP	32	SP	32	CR	13	SP	32
2	A	65	a	97	SOH	1	CR LF	13,10	0	48
3	B	66	b	98	STX	2	.SP	46,32	1	49
4	C	67	c	99	ETX	3	,SP	44,32	2	50
5	D	68	d	100	EOT	4	:SP	58,32	3	51
6	E	69	e	101	ENQ	5	!	33	4	52
7	F	70	f	102	AVK	6	"	34	5	53
8	G	71	g	103	BEL	7	#	35	6	54
9	H	72	h	104	BS	8	\$	36	7	55
10	I	73	i	105	HT	9	%	37	8	56
11	J	74	j	106	LF	10	&	38	9	57
12	K	75	k	107	VT	11	'	39	,	44
13	L	76	l	108	FF	12	(	40	.	46
14	M	77	m	109	CR	13	)	41	UL	
15	N	78	n	110	ESC	27	*	42	US	
16	O	79	o	111	FS	28	+	43		
17	P	80	p	112	GS	29	,	44		
18	Q	81	q	113	RS	30	-	45		
19	R	82	r	114	uS	31	.	46		
20	S	83	s	115	@	64	/	47		
21	T	84	t	116	\	92	:	58		
22	U	85	u	117	^	94	;	59		
23	V	86	v	118		95	<	60		
24	W	87	w	119	`	96	=	61		
25	X	88	x	120		124	>	62		
26	Y	89	y	121	~	126	?	63		
27	Z	90	z	122	DEL	127	[	91		
28	LL		US		LL		]	93		
29	ML		ML		UL		{	123		
30	DL		DL		PL		}	125		
31	BS		BS		BS		UL			

BS (Binary Shift) – переключение на ввод 8-битной строки символов с заданным размером. После BS следует 5-битное значение: если неравно нулю, то содержит в себе количество последующих байт строки; если 0, то следующие 11 бит содержат в себе количество байт минус 31. После окончания строки, кодирование возвращается в режим, который был до BS.

Символ, обозначенный FLG(n) в таблице 1.3, является специальным флагом, представляющим различные неинформационные символы, предоставляемые многими стандартными символиками. В битовом потоке значение FLG(n) сопровождается 3 дополнительными битами, кодирующими аргумент «n» в бинарном режиме, поэтому значение n может быть от 0 до 7.

FLG(0) представляет «FNC1» - флажок данных происходящий от Code 128. Когда FNC1 используется в первой позиции данных, это сигнализирует об использовании правил EAN/UPC формата данных, использующих Application Identifiers, и установке бита 0 в модификаторе идентификаторе символика. Когда FNC1 используется во второй позиции данных или в третьей позиции следуют 2 цифры, то это сигнализирует об использовании некоторого другого индустриально специфического формата, идентифицированного предыдущими данными и устанавливающим бит 1 в модификаторе. Когда FNC1 используется в дальнейших локациях, он служит в качестве разделителя области и вставляет ASCII код 29 в это место в выходных данных.

FLG(1) – FLG(6) предназначены для представления расширенного канального переходного символа ECE, который в выходной строке данных представлен как «\nnnnnn», наклонная черта влево, сопровождаемая 6 цифрами. Присутствие ECE где-нибудь в символе принуждает все наклонные черты влево внутри кодируемых данных быть удвоенным в выходной строке и также устанавливает

бит 2 в модификаторе идентификаторе символики. Аргумент «n» показывает, сколько из этих 6 цифр явно кодируются, используя режим Digit, в символе, заполненном нулями. ECE #000123, например, кодируется FLG(3), после чего кодирование возвращается в режим предшествующий FLG(n). FLG(7) – не используется.

На втором шаге кодирования сообщения результирующий поток бит делится на последовательность из B-битных ($B = 6, 8, 10$ или 12) кодовых слов. Анализируя последовательность, первые B-1 бит проверяются на наличие одинаковых бит информации. Если все B-1 бит равны 0, тогда добавляется бит пустышки (dummy), который равен 1, сразу после первых B-1 бит кодового слова, после чего следует последний бит и дальнейшие элементы последовательности. Если все B-1 бит равны 1, тогда добавляется бит пустышки (dummy), который равен 0. Это делается для того, чтобы «разбавить» возможные одинаковые биты в коде, чтобы не принесло пользы для чтения кода.

В конце, символ и границы байта в первоначальном сообщении не имеют никаких необходимых отношений с границами ключевого слова. Поэтому добавляются дополнительные пустышки с 1 (или 0 в случае необходимости), чтобы устранить любую двусмысленность. Возьмём пример кодирования фразы «Test code».

На рисунке 1.6. показано, как происходит перевод в биты информации и получения последовательности.

Перевод в биты		
ASCII символ	Значение из табл.	Биты
T	21	10101
L	28	11100
e	6	00110
s	20	10100
t	21	10101
SP	1	00001
c	4	00100
o	16	10000
d	5	00101
e	6	00110

Последовательность
 $10101+11100+00110+10100+10101+00001+00100+10000+00101+00110$
 Рисунок 1.6 – Перевод в биты информации и получение последовательности

Если используется одноуровневый «Aztec Code», то необходимо разбить последовательность по 6 бит. Затем проверить, необходимо ли добавлять dummy. Если нужно, то добавить (см. рисунок 1.7).

Перевод в блоки кодовых слов		Добавление dummy	
Блоки бит	Кодовое слово	Блоки бит	Кодовое слово
101011	43	101011	43
110000	48	110000	48
110101	53	110101	53
001010	10	001010	10
100001	33	100001	33
001001	9	001001	9
000000	0	000001	1
101001	41	010100	20
10		110111	55

Рисунок 1.7 – Перевод в кодовые слова и добавление dummy

После этого добавляются кодовые слова контроля ошибок, и генерируется код (см. рисунок 1.8).

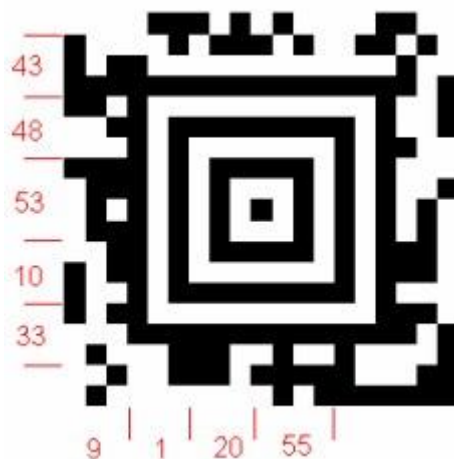


Рисунок 1.8 – 1-уровневый Aztec Code кодирующий фразу «Test Code»

Как видно из рисунка 1.8, значительную часть слоя занимают корректирующие кодовые слова Рида-Соломона.

Практическая часть

Задание 1.

1- Выбрать Aztec Code в соответствии с вариантом из прилагаемых к работе

Вариант	Код
Вариант 1	
Вариант 2	
Вариант 3	
Вариант 4	
Вариант 5	

Вариант 6		
Вариант 7		
Вариант 8		

2- Загрузите по ссылке программу https://disk.yandex.ru/d/T_HsWddTOmyl4Q VidikonReader и разберитесь с принципами работы.

3- Создайте копию примера кода (варианта задания) и в редакторе Paint случайным образом с помощью инструмента «Карандаш» зашумите данные кода. Зафиксируйте результаты распознавания в VidikonReader.

4- Создайте копию примера (варианта задания) и в редакторе Paint случайным образом с помощью инструмента «Линия» черного и белого цвета, зашумите всю мишень. Зафиксируйте результаты распознавания в VidikonReader.

5- Создайте копию примера (варианта задания) и в редакторе Paint случайным образом с помощью инструмента «Карандаш» зашумите область мишени. Зафиксируйте результаты распознавания в VidikonReader.

6- Повторите шаги 3 – 5 с различными параметрами корректировок.

Задание 2. Используя теорию кодирования кода Aztec вручную (без использования сторонних программ) определите закодированную фразу.

Вариант	Фраза
Вариант 1	Good morning
Вариант 2	Good afternoon
Вариант 3	Good evening
Вариант 4	Thank you very much
Вариант 5	See you later
Вариант 6	Have a nice day
Вариант 7	As far as I know
Вариант 8	As far as I remember

Перечень контрольных вопросов

- 1- Что понимается под дискретизацией графической информации?
- 2- В каких единицах измеряется разрешение (ppi, dpi)?
- 3- Как разрешение влияет на качество и детализацию изображения?
- 4- В чём заключается процесс квантования изображения?
- 5- Сколько цветов можно закодировать при глубине 8 бит, 16 бит, 24 бита?
- 6- От чего зависит размер цифрового изображения в памяти?